

虫虫家装 AI 技术报告

1. 项目概述与参赛说明

《虫虫家装》是一款以“家园装修 + 野外探索 + 情感陪伴”为核心体验的原创游戏作品。玩家在探索废墟与自然遗迹的过程中收集材料、家具与配饰，为小动物们布置家园，并通过持续互动建立关系，推进剧情与成长。

本次参赛版本聚焦 AI 技术在游戏中核心体验中的深度整合。作品当前未商业化，并随参赛材料提交完整可运行版本。本文档用于说明本作品所采用的 AI 技术、模型接入方式、系统架构及其在游戏中的具体实现。

从赛题要求对应来看，本项目的 AI 应用主要覆盖以下方向：

- 智能 NPC：让小动物依据玩家行为、关系阶段与记忆状态生成个性化反馈。
- 玩家体验个性化：让同一互动事件在不同上下文中呈现不同回应，形成持续变化的陪伴体验。
- 程序化内容生成：以事件驱动的方式生成与当前游戏状态绑定的叙事文本内容。

2. 为什么 AI 让这款游戏更好玩

2.1 AI 让“陪伴感”从静态文本变成可成长的关系体验

《虫虫装修》的核心乐趣，不只是收集家具和完成装修，而是让玩家在帮助小动物“拥有一个家”的过程中，逐步建立真实的情感连接。如果 NPC 始终只使用固定台词，那么玩家的付出只能得到重复反馈，关系成长会停留在数值层面，难以形成真正的陪伴感。

AI 的引入，使小动物能够根据当前信任度、长期记忆、近期经历和玩家行为，给出符合关系阶段的自然语言回应。这样一来，玩家感受到的不是“触发了一条预设对话”，而是“它记得我之前做过什么，也会因为我们现在的关系不同而说出不同的话”。

2.2 AI 让核心玩法形成完整闭环

本项目的核心玩法闭环是以家园为中心展开的。在这个闭环里，玩家的装修行为既会带来经济系统和数值系统上的反馈，也会带来小动物的情绪与关系反馈。完整链路可以概括为：

玩家探索并收集资源 -> 为小动物布置和改造家园 -> 获得经济与数值上的成长反馈 -> 触发小动物的情绪与关系反馈 -> 推动玩家继续投入互动、装修与经营。

在这个闭环中，数值反馈当然重要，它能为玩家提供成长确认和系统回报；但相比单纯的收益变化，本项目更看重情感反馈所带来的驱动力。原因在于，家园装修本身并不是一个纯功利性的玩法，它天然带有很强的审美属性和创作属性。玩家并不只是为了“涨数值”才去装修，更是在通过布置空间表达偏好、投入情感，并主动创造一个自己愿意停留、也希望角色喜欢的家。

AI 在这里承担的是“把玩家的创作行为转化为情感反馈”的关键角色。送礼、靠近、抚摸、摆放喜欢的家具、移走不合适的家具，这些原本可能只会落在数值层的操作，在 AI 驱动下变成了可以被角色表达、记住、解释和回应的互动事件。这样，装修就不再只是视觉变化或收益提升，而是会被角色真正“感受到”的行为；情感系统也不再只是 UI 上的好感度，而会被玩家直接听见、看见、体验到。对本项目而言，经济与数值反馈负责提供结构和回报，AI 驱动的情感反馈则负责把这套玩法真正变得动人。

2.3 AI 让重复互动保持新鲜感

模拟经营与陪伴类游戏天然面临一个问题：玩家的日常互动动作会不断重复。如果每次触发反馈都相同，体验会很快进入机械重复。

本项目通过将角色设定、关系阶段、世界事件与短期记忆注入生成过程，使同一类触发在不同时间点产生差异化回应。例如，玩家在野外死亡后狼狈回家，紧接着又为小动物做了一次大规模装修，角色的反馈就不应只是简单夸奖“你今天很勤劳”。更理想的反馈链条是，角色先对玩家此前的遭遇表示心疼和关心，再把这次大装修和之前的冒险联系起来，例如表达“你刚刚明明那么辛苦，回来以后却第一时间在帮我布置家。你出去收集那些东西，原来真的是为了给我一个更好的家吗？”

这种带有因果链条的反馈，比单点表扬更能体现“它记得我，也理解我刚刚经历了什么”。对玩家来说，动人的地方不只是角色会说话，而是角色会把多个行为串起来理解，从而让陪伴关系显得更真实、更有沉浸感。

3. AI 在游戏中的深度整合位置

本项目的 AI 不是一个独立于主玩法之外的聊天功能，也不是可有可无的展示性模块，而是嵌入在“玩家行为 -> 角色状态变化 -> 角色表达反馈 -> 关系推进”的核心循环中。

具体来说：

- 玩家在野外探索和家园经营中的关键动作，会触发角色侧的事件记录与状态变化。
- 这些状态变化会进入小动物的即时状态、短期记忆和长期关系信息中。
- AI 根据这些结构化状态生成角色表达，使反馈与当前关系阶段、角色个性和世界上下文保持一致。

- 角色表达反过来强化玩家对互动结果的感知，推动玩家继续投入收集、装修和陪伴。

因此，AI 在本项目中承担的不是“额外增加一层文本表现”，而是让核心玩法从“完成操作”升级为“建立关系”的关键机制。这也是本作品满足“深度整合 AI 技术”的主要依据。

4. 技术实现总览

4.1 系统架构分层

本项目的运行时逻辑分为三层：

1. 规则与模拟层 负责需求衰减、动机与行为、信任数值变化、世界事件维护、短期事件维护等核心逻辑。该层在没有 LLM 的情况下仍可完成游戏的基础闭环，保证整体系统的稳定性和可降级性。
2. 叙事与表达层 在特定交互事件触发时，根据当前游戏状态快照调用大语言模型，生成符合角色设定和当前关系阶段的中文台词。该层负责情感表达增强，不直接负责行为决策。
3. 表现层 通过全屏对话界面和非阻塞气泡将生成结果反馈给玩家，并配合加载状态、请求去重、过期时间和冷却机制，确保运行时体验流畅。

这一分层方式使得本项目既能保持 AI 表现的灵活性，又不会将底层玩法逻辑交给不可控的黑盒模型处理。

4.2 所用模型与接入方式

本项目采用兼容 OpenAI Chat Completions 协议的大语言模型接口，并保留通过环境变量切换上游模型地址与模型名的能力。参赛版本实际采用的模型为 google/gemini-2.5-flash-lite。

在模型选型上，本项目优先考虑响应速度，而不是追求复杂长对话能力。原因很明确：在游戏场景中，反馈是否及时，往往比单次回复是否“更聪明”更直接影响体验。玩家完成一个动作后，如果需要等待较长时间才能看到角色反应，那么情绪反馈链路就会被打断。因此，参赛版本优先选择 google/gemini-2.5-flash-lite 这类强调速度与成本效率的轻量模型，以换取更短的响应时间和更稳定的触发体验。对本项目而言，运行时任务并不是开放式长篇对话，而是围绕单次事件生成短反馈、关系化反馈和角色化反馈，这类模型已经足以覆盖这一目标。

模型调用采用前后端分离的安全接入方式：

- 若前端环境显式配置模型密钥，可按配置直连模型接口。

- 在正式部署与参赛交付场景中，推荐由服务端代理持有并注入密钥，前端通过同源接口发起请求，避免将敏感信息打包进客户端。

这种实现兼顾了开发效率、部署灵活性与安全性，也便于后续根据赛场环境、网络条件和延迟目标替换不同模型提供方。

4.3 结构化上下文构建

为了避免自由聊天场景中常见的上下文漂移问题，本项目并不直接把长聊天历史拼接后交给模型，而是把当前游戏状态组织为结构化摘要后再注入提示词。主要包括以下信息：

- 角色档案：名称、物种、性格文本、偏好列表等。
- 即时状态：需求水平、当前活动状态等。
- 长期记忆：信任度、对玩家的印象、重要事件、认知总结等。
- 短期记忆：近期行为事件的时间线摘要。
- 世界事件：按玩家行动、该角色行动和其他事件分类的最近事件序列。

这种设计带来三方面价值：

- 提高可控性：生成内容更容易锚定到当前游戏状态。
- 提高可解释性：开发团队可以明确知道模型“看到了什么”。
- 提高个性化程度：同一触发事件在不同关系背景下可产生不同输出。

4.4 以玩家行为为中心的事件驱动机制

本项目的 Prompt 工程首先服务于一个产品层判断：AI 必须优先对玩家的行动做出即时反馈。我们不把 AI 当作一个等待玩家“主动开口”的聊天对象，而是把它设计成一个会对玩家操作持续作出反应的反馈系统。

因此，模型调用并不是随时发生，而是由明确的游戏事件触发。当前设计中，典型触发包括：

- 玩家主动靠近小动物。
- 玩家送出礼物。
- 玩家抚摸角色。
- 玩家摆放与角色偏好相关的家具。
- 玩家移走相关家具或改变环境。

每种触发都会对应专属的场景描述和提示约束，使模型生成内容始终围绕具体互动展开。这种事件驱动机制本质上是一种与玩法动作深度绑定的程序化叙事生成方式。

它的核心不是“让 AI 多说话”，而是让 AI 对玩家刚刚做过的事立刻产生反馈，从而把玩家操作、角色反应和关系推进连成一条完整链路。

4.5 交互设计的减法原则

在交互层面，本项目刻意没有设计“玩家直接向 AI 发起自由对话”的主流程，无论是文字输入还是语音输入，都不作为核心玩法入口。

这样做有两个原因：

1. 降低操作负担。对于一款以探索、装修和陪伴为核心的游戏来说，要求玩家频繁打字或长时间说话，会把原本轻量、连续的互动体验变成额外的心理负担。玩家本来是在和小动物相处，结果却变成了不断为 AI 组织语言。
2. 把反馈自动化。传统游戏设计里，好的反馈往往是在玩家完成动作后自动到达，而不是要求玩家再额外做一层输入。本项目延续这一原则：玩家只需要专注于玩游戏，AI 则在后台根据玩家行为自动生成情绪反馈。这样既保留了游戏交互的轻量感，也发挥了 AI 在个性化表达上的优势。

项目初期我们确实考虑过把自由对话作为重要能力，希望用更高的交互自由度来增强真实感和情感链接。但在后续判断中我们发现，一旦把聊天机器人式的自由问答放进游戏主循环，它很容易喧宾夺主，让玩家的注意力从探索、装修和陪伴关系本身偏移到“怎么和 AI 聊天”这件事上。与此同时，这类纯聊天体验能带来的持续乐趣有限，且市场上已经出现较强的同质化趋势。如果继续沿着聊天机器人方向强化，反而会破坏游戏整体的设计平衡。

这一减法原则直接影响了 Prompt 工程的组织方式。既然玩家不直接向 AI 提问，那么模型输入就必须高度依赖“玩家做了什么”“角色当前怎么看待玩家”“此刻世界里正在发生什么”这三类信息。

4.6 Prompt 工程设计

本项目的生成过程采用两段式消息结构：

- **System Prompt**：注入角色设定、当前活动、需求描述、关系阶段、长期记忆、世界事件与语气约束，规定输出风格、长度和表达边界，并要求角色必须使用第一人称、严格控制输出长度、不得脱离当前角色身份。
- **User Prompt**：绑定当前触发事件，明确说明此刻发生了什么，让模型围绕当前互动进行第一人称表达。

这种 Prompt 组织方式的目标不是让模型“自由发挥”，而是要求它回答一个更具体的问题：玩家刚刚做的这件事，对这个角色来说意味着什么。

在约束层面，本项目特别强调三条硬规则：

- 必须使用第一人称，保证玩家感知到的是角色自身的主观表达，而不是系统说明。
- 严格限制输出长度，确保反馈更接近游戏内即时反应，而不是冗长发言。
- 严格遵守事实边界，不能假设或编造记忆中没有发生过的事情，必须基于真实交互历史和最近一次触发事件做出回应。

从 Prompt 设计角度看，这种做法带来两个直接收益：

- 提升用户存在感。玩家真正需要的不是抽象的聊天能力，而是“我刚刚做的事被看见了”。如果角色输出与玩家本人、玩家操作或当前事件没有关系，再流畅的句子也会显得空。
- 提升内容可控性。把生成范围限制在当前事件、角色设定和记忆上下文内，可以明显减少无关发散、言之无物和逻辑漂移的问题，使反馈更稳定、更像游戏内角色在说话，而不是一个泛用聊天机器人在即兴发挥。

4.7 一致性校验与降级策略

为了提升生成结果的可信度与可用性，本项目在主生成之外增加了轻量级一致性校验机制。这样做的核心原因不是追求技术复杂度，而是尽可能压低“幻觉”出现的概率。

生成管线如下：

1. 首次调用模型生成角色台词。
2. 再次调用模型，对该台词是否与已记录事件矛盾进行判别。
3. 如果校验失败，则进行有限次数的重新生成。
4. 如果仍未得到有效结果，则回退到预设模板文案。

如果角色说出了游戏里没有发生过的事，或者把事实关系说反，玩家的沉浸感会被立刻破坏，情感链接也会迅速断裂。因此，本项目宁愿让系统多调用几次模型，甚至在逻辑冲突时保持沉默，也不希望角色说出离谱、失真的内容。

这套机制的核心目标是平衡三件事：

- 保留生成式 AI 的表现力。
- 降低与事实冲突的概率。
- 确保在超时、失败或异常情况下，游戏表达层不会断裂。

也就是说，本项目并没有把生成结果无条件直接展示给玩家，而是通过“生成 -> 校验 -> 重试 -> 模板兜底”的工程管线，将 AI 纳入一个可运行、可演示、可交付的游戏系统中。

4.8 性能、安全与运行时保障

为保证 AI 系统可以稳定服务于实时游戏体验，本项目额外实现了以下工程机制：

- 服务端密钥代理：避免模型 API Key 在前端暴露。
- 超时控制：为模型调用设置超时阈值，避免长时间阻塞。
- 请求去重：防止同一类请求并发堆积。
- 触发冷却：针对高频事件设置冷却时间，防止请求风暴。
- 异步非阻塞展示：气泡类反馈在后台执行，不阻塞主循环。
- 模板兜底：在模型不可用时仍保证角色有基本反馈能力。

这些措施共同保证 AI 能够作为游戏运行时系统的一部分长期存在，而不是仅在演示环境中短暂可用。其中，低延迟优先、小模型优先、事件触发优先于自由对话的设计取向，决定了本项目更像一个“实时反馈系统”，而不是一个追求长篇闲聊能力的 AI 陪聊产品。

5. 具体体验示例

为了更直观地说明 AI 如何参与核心玩法，下面以“家具互动”作为一个典型示例：

玩家在探索中获得一件符合某只小动物偏好的家具，并将其摆放到家中。系统首先记录该行为事件，更新世界事件时间线与角色短期记忆；如果该家具与角色偏好匹配，角色当前关系状态与认知上下文会一并进入本次生成输入。随后，AI 不只是返回一句泛泛的“谢谢”，而是依据角色性格、信任度和近期经历，生成更具体的反馈。例如，角色可能会提到这个布置让它想起之前玩家帮助它整理房间的经历，或者表达“原来你真的记得我喜欢这种风格”。

这时，玩家感知到的不是“装修数值上涨了”，而是“我的装修选择被角色理解并记住了”。这正是 AI 对玩法体验的核心提升。

6. 与赛题要求的对应关系

6.1 关于“深度整合 AI 技术”

本项目中的 AI 并非独立展示模块，而是直接作用于家园装修、送礼、靠近、抚摸等核心玩法事件，并参与角色反馈、关系成长和情感陪伴体验的形成，因此属于对核心玩法循环的深度整合。

6.2 关于“智能 NPC”

小动物 NPC 的表达并非固定脚本，而是由角色设定、关系阶段、记忆状态与当前事件共同驱动生成，能够体现不同角色的个性与情境差异，符合智能 NPC 的赛题方向。

6.3 关于“玩家体验个性化”

同样的玩家行为在不同游戏进度、不同信任度和不同记忆背景下会获得不同回应，因此玩家体验并非统一模板，而是随互动历史逐步分化，符合玩家体验个性化的赛题方向。

6.4 关于“程序化内容生成”

本项目当前主要采用事件驱动的叙事文本生成，程序化生成对象聚焦于角色台词与情感反馈内容，而不是地图、关卡或任务图本身。该策略与项目当前的玩法重点保持一致，也让 AI 应用更集中地服务于“陪伴式家园体验”。

7. 研发与制作流程中的 AI 应用

除运行时 AI 外，团队也在研发与制作流程中使用了 AI 辅助工具，但这一部分属于辅助生产能力，不替代游戏内 AI 的核心价值。

7.1 AI 辅助编程

团队在开发过程中深度使用 AI 辅助编程。以主程为例，其编写代码中约 99% 有 AI 辅助参与；从团队整体看，平均 AI 辅助代码占比也在 90% 以上。AI 主要用于代码生成、方案比较、重构讨论和文档整理，以提高接口设计与工程迭代效率。所有最终合入项目的代码、架构决策与系统行为仍由人工确认、测试和把关。

7.2 AIGC 美术与过场辅助

项目在部分美术过场和演出素材制作中使用了 AI 生成式图像或视频能力。其中，过场动画使用了 Seed-Dance 模型生成，并由团队进一步完成筛选、剪辑、调色、合成和节奏编排，使其服务于游戏氛围和叙事演出。

此外，拼贴画风的家具贴图和纹理图片也由 AI 生成，但家具的形状、结构和透视关系仍由美术人员手绘把控。原因在于，现阶段 AI 在固定视角和统一透视的家具素材生成上仍不够稳定，直接全交由模型处理会影响成品一致性。

7.3 动画制作辅助

部分角色或界面动画在制作中使用 AI 辅助进行动作参考和关键动作过程帧生成。例如，团队会先利用 AI 生成角色从站立到躺下等动作过程中的中间图，再由动画师基于这些素材进行实际制作和整理。这种方式在美术人力有限的条件下，显著提升了动画岗位与其他岗位之间的协作效率。最终表现结果仍由人工在工具链中定稿，并与游戏逻辑进行绑定。

8. 项目特点与技术边界

本项目的特点不在于训练了一个专用模型，而在于在不进行模型微调的前提下，通过结构化状态注入、事件驱动触发、一致性校验与多级降级策略，把通用大语言模型有效地纳入了一个真实可运行的游戏交互系统。

同时，我们也明确划定了技术边界：

- 角色行为决策和数值演化由规则系统负责，不直接交给 LLM 决策。
- LLM 主要负责情感表达和叙事反馈，不替代底层玩法逻辑。
- 研发流程中的 AI 工具使用，与运行时玩家可感知的 AI 体验属于不同层级，文档中分开描述以避免混淆。

这种边界划分使系统在表现力、稳定性和工程可控性之间取得了平衡。

9. 总结

《虫虫家装》的 AI 设计目标，不是单纯让 NPC 更会说话，而是让“探索 - 装修 - 陪伴 - 关系成长”这条核心玩法链路真正成立。

通过将角色设定、即时状态、长期记忆、短期事件与世界事件结构化注入大语言模型，并配合事件驱动触发、一致性校验、请求冷却、异步展示与模板兜底等机制，本项目实现了一个既能体现角色个性、又能稳定运行的智能 NPC 表达系统。

最终，AI 让小动物不再只是等待玩家装修的对象，而成为会记住玩家、会随着关系变化而表达情绪、并能持续回应玩家付出的陪伴者。这正是 AI 为本项目带来的核心玩法增益，也是本作品参加 AI 游戏赛道的核心价值所在。